Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

5. Giao tiếp bất đồng bộ — Kafka, Events & Messaging . . . 2
5.1. Động lực của giao tiếp bất đồng bộ . . . 3
5.1.1. Vấn đề mà sync không giải quyết được . . . 3
5.1.2. Messaging patterns . . . 4
5.2. Message Broker — hai trường phái thiết kế . . . 5
5.2.1. Durable vs Ephemeral Brokers . . . 5
5.2.2. RabbitMQ — Smart Routing & Flexible Messaging . . . 7
5.2.3. So sánh chi tiết . . . 7
5.2.4. Khi nào dùng gì? . . . 8
5.3. Apache Kafka — kiến trúc chi tiết . . . 9
5.3.1. Các khái niệm cốt lõi . . . 9
5.4. Producer/Consumer Pattern trong KBM . . . 10
5.4.1. Kafka Producer . . . 10
5.4.2. Kafka Consumer . . . 11
5.4.3. Luồng hoàn chỉnh . . . 12
5.5. Delivery Guarantees & Idempotency . . . 13
5.5.1. Ba cấp độ đảm bảo . . . 13
5.5.2. Idempotency — thiết kế để chịu được duplicate . . . 13
5.6. Event Schema Design . . . 14
5.6.1. Cấu trúc một event . . . 14
5.6.2. Bốn loại message . . . 15
5.6.3. Schema evolution . . . 15
5.7. Kafka vs DB Queue — chọn broker theo bài toán . . . 16
5.8. WebSocket — Real-time Notifications . . . 16
5.8.1. Bài toán . . . 16
5.9. Case study: KBM . . . 17
5.9.1. Bài toán kỳ thi . . . 18
5.9.2. Kiến trúc pipeline 2 Kafka topic . . . 18
5.9.3. Phân tích các vấn đề . . . 18
5.9.4. Lộ trình củng cố pipeline . . . 19
5.10. Tổng kết . . . 20
5.11. Đọc thêm . . . 20
Tài liệu tham khảo . . . 21

CHƯƠNG 5

Giao tiếp bất đồng bộ — Kafka, Events & Messaging

> "Event streams become the heart of data sharing throughout the company. Data no longer sits solely on a database and is accessible only through synchronous interfaces; instead, it is shared in event streams readily available for every current and future consumer."
>
> — Hugo Rocha, *Practical Event-Driven Microservices Architecture* [5]

MỤC TIÊU HỌC TẬP

- Hiểu tại sao giao tiếp bất đồng bộ giải quyết các giới hạn của giao tiếp đồng bộ
- Phân biệt hai loại message broker: durable (Kafka) vs ephemeral (RabbitMQ)
- Nắm vững kiến trúc Apache Kafka: topic, partition, consumer group
- Hiểu kiến trúc RabbitMQ: exchange, queue, binding
- Thiết kế event schema: 4 loại message (Command, Event, Document, Query)
- So sánh các cấp độ đảm bảo giao nhận: at-most-once, at-least-once, exactly-once
- Sử dụng WebSocket cho thông báo thời gian thực
- Phân tích Kafka pipeline và DB queue trong KBM

5.1. Động lực của giao tiếp bất đồng bộ

Giống như việc sinh viên nộp đơn vào hòm thư thay vì đứng chờ trực tiếp trước phòng giáo vụ, giao tiếp bất đồng bộ cho phép hệ thống tiếp nhận yêu cầu và xử lý sau, giải phóng các dịch vụ khỏi sự chờ đợi chéo nhau gây tắc nghẽn.

5.1.1. Vấn đề mà sync không giải quyết được

Chương 4 đã phân tích ba vấn đề cốt lõi của giao tiếp đồng bộ: sự phụ thuộc về mặt thời gian (temporal coupling), lỗi dây chuyền (cascading failure), và độ trễ cộng dồn. Giao tiếp bất đồng bộ (*asynchronous messaging*) làm nhẹ cả ba bằng cách **tách rời** (*decouple*) producer và consumer qua một trung gian — message broker.



TIÊU CHÍ	ĐỒNG BỘ (REST / GRPC)	BẤT ĐỒNG BỘ (KAFKA / RABBITMQ)
Độ trễ	thấp – phản hồi trực tiếp	cao hơn – đi qua broker
Coupling	cao – A phụ thuộc trực tiếp B	thấp – A không cần biết B
Khi có lỗi	lan dây chuyền	khoanh vùng tại B
Thông lượng	giới hạn bởi tốc độ của B	broker đệm – mở rộng độc lập
Độ phức tạp	đơn giản – request/response rõ ràng	phức tạp hơn (idempotency, thứ tự)

Hình 5.1: *So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled)*

Thay vì bị khóa cứng về mặt thời gian, giao tiếp bất đồng bộ làm nhẹ phần lớn những điểm yếu này. Với bài toán **Temporal coupling**, producer chỉ cần gửi thông điệp vào broker rồi tiếp tục công việc khác, trong khi consumer có thể xử lý sau khi đã sẵn sàng. Với **Cascading failure**, phạm vi ảnh hưởng được thu hẹp: nếu một service tạm ngưng hoạt động, thông điệp vẫn nằm tại broker chờ đến khi service đó phục hồi, thay vì đẩy lỗi ngược lên phía gọi. Cần lưu ý rằng rủi ro không biến mất mà dịch chuyển — hàng đợi có thể tồn đọng và chính broker trở thành thành phần phải được bảo vệ. Về **độ trễ**, thay vì bắt người dùng phải chờ đợi hệ thống xử lý xong toàn bộ chuỗi tác vụ, ứng dụng có thể trả về phản hồi ngay lập tức và gửi kết quả thực tế qua cơ chế thông báo hoặc polling sau đó.

Broker thay vì Async HTTP

Nhiều lập trình viên mới thắc mắc: *"Tại sao không gửi một HTTP request và không cần chờ kết quả (fire-and-forget)?"*

Câu trả lời là: Giao tiếp HTTP bất đồng bộ giống như một cuộc gọi điện thoại — dù người gọi truyền đạt thông tin rất nhanh rồi ngắt kết nối, người nhận vẫn bắt buộc phải đang trong trạng

thái sẵn sàng tiếp nhận (online) ngay tại thời điểm đó. Còn Message Broker giống như việc *sinh viên nộp đơn phúc khảo vào hộp thư Phòng Đào tạo*. Người gửi bỏ đơn vào hộp (Broker), hệ thống cam kết sẽ giữ đơn và chuyển đến tay chuyên viên, kể cả khi máy chủ người nhận đang gặp sự cố hay mất điện.

Richardson trong [2a, Ch.3] phân tích: async messaging cải thiện **tính sẵn sàng (availability)** vì các dịch vụ không phụ thuộc lẫn nhau tại runtime. Tuy nhiên, hệ thống phải đánh đổi bằng **độ phức tạp (complexity)** — tính nhất quán cuối cùng (eventual consistency), thứ tự thông điệp (message ordering) và xử lý trùng lặp (duplicate handling) — đây là những chi phí không thể tránh khỏi.

Nguyên tắc dưới đây tóm tắt sự đánh đổi đó.

Đổi tính nhất quán lấy tính sẵn sàng

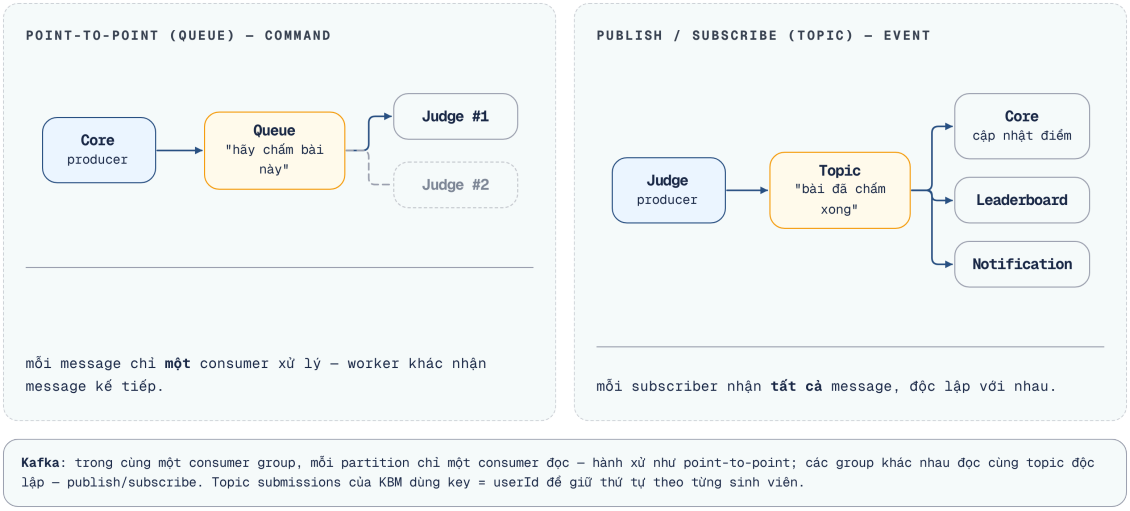
Async messaging cải thiện tính sẵn sàng và giảm temporal coupling — các dịch vụ không cần online cùng lúc — nhưng phải trả giá bằng tính nhất quán cuối cùng, khử trùng lặp (deduplication) và xử lý dead letter. Đây là sự đánh đổi có chủ đích, không phải hệ quả ngẫu nhiên. Trước khi chọn async, câu hỏi bắt buộc: "Khoảng thời gian chờ nhất quán (consistency window) bao lâu là chấp nhận được cho bài toán thực tế này?"

5.1.2. Messaging patterns

Có ba messaging pattern cơ bản [2a, Ch.3]:

1. **Point-to-point (Queue)** — Một message, một consumer. Giống như giảng viên giao đề tài riêng cho một sinh viên trên hệ thống LMS (sinh viên này đã nhận và xử lý thì người khác không cần làm nữa). Phù hợp cho command: "hãy chấm bài này".
2. **Publish/Subscribe (Topic)** — Một message, nhiều consumer. Giống như Phòng Đào tạo gửi thông báo toàn trường (bất kỳ phòng ban nào quan tâm đều tiếp nhận được thông tin nguyên bản độc lập với nhau). Phù hợp cho event: "bài đã được chấm xong".
3. **Request/Async Response** — Gửi request qua messaging, nhận response qua message khác (có correlation ID). Kết hợp lợi ích của cả hai.

Hình 5.2 so sánh hai mô hình phân phối đầu tiên.



Hình 5.2: Point-to-Point (Queue) vs Publish/Subscribe (Topic)

Command vs Event

Phân biệt rõ **command** (yêu cầu hành động: "ChấmBài", "GửiEmail") và **event** (thông báo sự kiện đã xảy ra: "BàiĐãĐượcChấm", "ĐiểmĐãCậpNhật"). Các lệnh (command) thường được phân phối theo mô hình point-to-point, trong khi các sự kiện (event) thường tuân theo mô hình pub/sub. Sự phân biệt này ảnh hưởng đến thiết kế schema và xử lý lỗi [5, Ch.8].

5.2. Message Broker — hai trường phái thiết kế

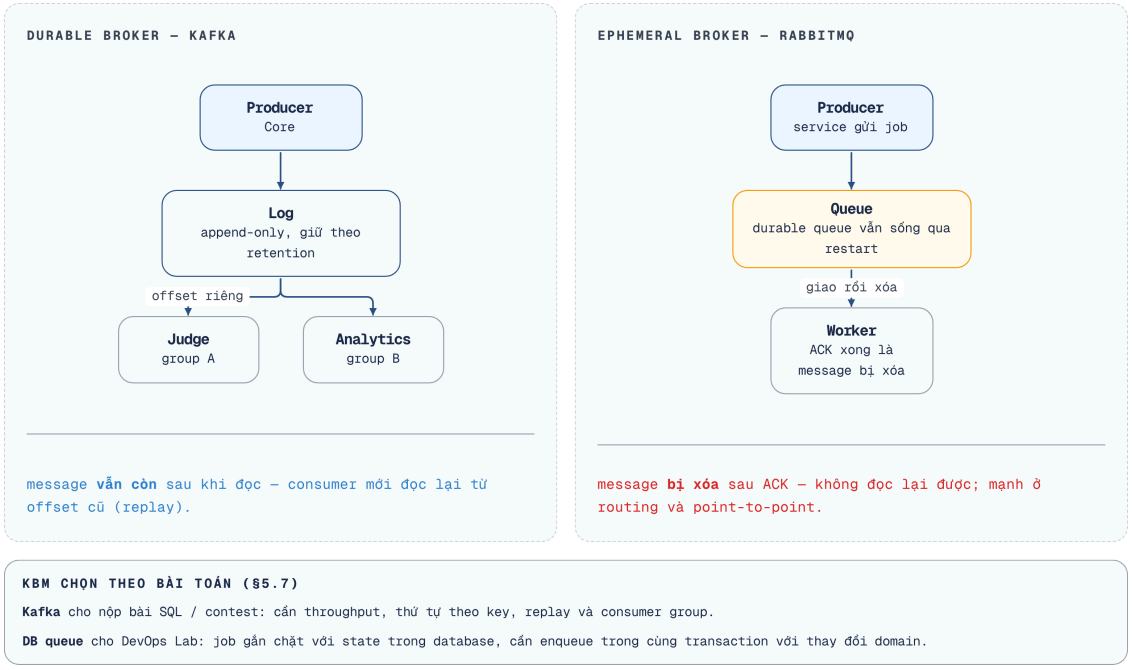
Message Broker hoạt động giống như trung tâm bưu điện của trường đại học. Tùy thuộc vào cách thức lưu trữ và phân phát thư tín, các thiết kế được chia thành hai trường phái trái ngược, quyết định trực tiếp đến sức chịu tải của toàn bộ hệ thống.

5.2.1. Durable vs Ephemeral Brokers

Rocha trong [5, §3.1.3] phân loại message broker thành hai trường phái cơ bản dựa trên vòng đời của message:

- **Ephemeral (tạm thời)**: Thông điệp bị xóa sau khi consumer gửi phản hồi xác nhận (acknowledge). Đại diện: **RabbitMQ**, ActiveMQ, Amazon SQS.
- **Durable (bền vững)**: Thông điệp được lưu trữ trên ổ cứng (disk) và vẫn sẵn sàng (available) sau khi được tiêu thụ (consume). Đại diện: **Apache Kafka**, Apache Pulsar, Amazon Kinesis. Hệ sinh thái này cũng xuất hiện các lựa chọn mới đáng chú ý: **Redpanda** (tương thích Kafka API nhưng viết bằng C++, một binary duy nhất, không cần JVM — gọn nhẹ cho self-hosted) và **NATS JetStream** (nhẹ, độ trễ thấp, phù hợp hệ thống nhỏ). Nội dung chương dùng Kafka làm đại diện; các khái niệm partition, offset, consumer group áp dụng gần như nguyên vẹn cho Redpanda.

Hình 5.3 minh họa khác biệt giữa hai trường phái: message bị xóa sau khi được xác nhận, hay vẫn được giữ lại để đọc lại.



Hình 5.3: Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn)

Ở đây, "ephemeral" chỉ việc message bị xóa sau khi được xác nhận tiêu thụ (destructive consumption). Thuật ngữ này không có nghĩa RabbitMQ thiếu độ bền: durable queue và persistent message vẫn sống sót qua lần khởi động lại. Ngược lại, "durable" theo nghĩa của Kafka là message được giữ lại theo chính sách lưu giữ (retention) để đọc lại (replay), chứ không mặc nhiên vĩnh viễn.

Sự khác biệt này **không chỉ là kỹ thuật** — nó ảnh hưởng đến cách thiết kế hệ thống. Các broker lưu trữ bền vững (durable broker) cho phép phát lại (replay), lưu vết sự kiện (event sourcing), và hỗ trợ nhiều bên tiêu thụ (consumer) cùng đọc một luồng dữ liệu. Trong khi đó, các broker trung chuyển (ephemeral broker) tập trung vào định tuyến thông minh (smart routing), hàng đợi ưu tiên (priority queue), và giao tiếp điểm-điểm (point-to-point communication).

Chọn broker theo bài toán thực tế

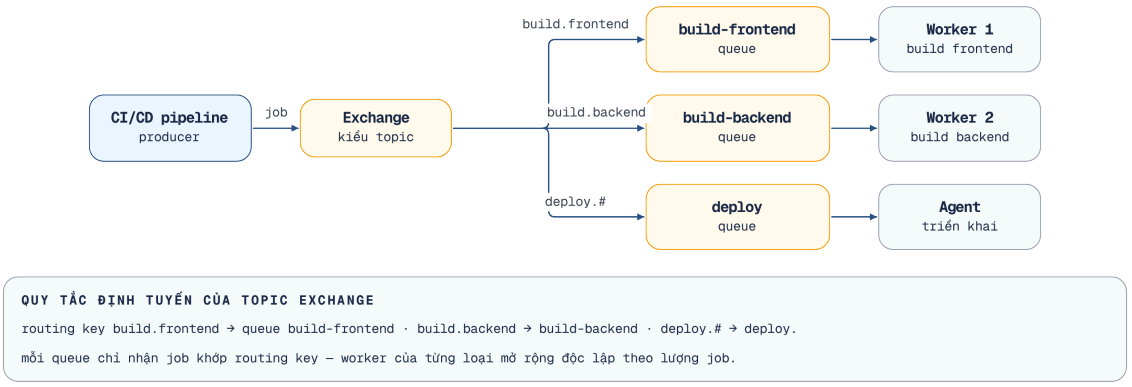
Durable broker (Kafka) phù hợp khi cần replay và nhiều consumer cùng đọc (Kafka có thể làm kho sự kiện cho event sourcing, kèm các giới hạn ở Chương 7). Ephemeral broker (RabbitMQ) phù hợp khi cần smart routing và point-to-point delivery. Chọn broker dựa trên **thói quen của đội ngũ phát triển** thay vì đặc điểm kỹ thuật sẽ dẫn đến sự lệch pha (mismatch) kiến trúc khó sửa chữa về sau.

Việc lựa chọn broker giống như việc thiết lập kênh thông báo trong trường đại học. Nếu cần lưu trữ toàn bộ lịch sử điểm số để sinh viên có thể tra cứu lại bất cứ lúc nào (lưu vết sự kiện), Kafka là hệ thống sổ cái đào tạo phù hợp nhất. Ngược lại, nếu chỉ cần phân luồng đơn từ phúc khảo đến đúng phòng ban xử lý và có thể bỏ đi sau khi giải quyết xong, RabbitMQ hoạt động như một hòm thư tiếp nhận thông minh.

5.2.2. RabbitMQ — Smart Routing & Flexible Messaging

RabbitMQ dựa trên **AMQP** (Advanced Message Queuing Protocol), với kiến trúc phong phú hơn Kafka về routing:

Trong môi trường đại học, không phải mọi thông báo đều được phát loa toàn trường. RabbitMQ cung cấp một cơ chế định tuyến thông minh (smart routing), giống như một nhân viên phân loại thư tín tại Phòng Hành chính, đảm bảo mỗi đơn từ hay yêu cầu nghiệp vụ đều được chuyển phát chính xác đến tay người cần xử lý.



Hình 5.4: Kiến trúc RabbitMQ — Exchange định tuyến message đến các queue

Khác với mô hình append-only log đơn giản của Kafka, message trong RabbitMQ đi qua exchange trước khi vào queue.

Dưới đây là các khái niệm cốt lõi của RabbitMQ:

- **Exchange**: Nhận message và định tuyến (route) đến các queue theo các quy tắc thiết lập trước.
- **Queue**: Lưu trữ message trong khi chờ consumer tiêu thụ (hoạt động theo nguyên tắc FIFO).
- **Binding**: Quy tắc kết nối từ exchange đến queue (thông qua routing key, header).
- **Ack/Nack**: Cơ chế để Consumer xác nhận đã xử lý xong (ack) hoặc từ chối (nack → requeue/dead letter).
- **Exchange types**: Direct, Fanout, Topic, Headers — mỗi loại cung cấp một logic định tuyến riêng biệt.

Trong khi Kafka chú trọng vào khả năng lưu trữ bền vững và thông lượng cực lớn bằng cách phân vùng dữ liệu đơn giản theo partition, thì RabbitMQ lại có thế mạnh ở năng lực **smart routing**. Một thông điệp trong RabbitMQ có thể được nhân bản, lọc hoặc định tuyến phức tạp đến đúng các hàng đợi mục tiêu dựa trên routing key, cấu trúc header matching, hoặc broadcast tới nhiều thành phần của hệ thống.

5.2.3. So sánh chi tiết

Dưới đây là so sánh chi tiết giữa hai hệ thống này theo từng tiêu chí cụ thể:

- **Loại hệ thống**: Kafka là một durable event log, lưu trữ lâu dài. RabbitMQ là ephemeral message queue, xóa thông điệp sau khi xử lý.
- **Mô hình dữ liệu**: Kafka sử dụng cấu trúc append-only log, nơi consumer theo dõi qua offset. RabbitMQ sử dụng hàng đợi FIFO, thông điệp biến mất sau khi được xác nhận (ack).
- **Throughput (Thông lượng)**: Kafka đạt mức rất cao (hàng triệu thông điệp mỗi giây). RabbitMQ với cấu hình truyền thống ở mức hàng chục nghìn thông điệp mỗi giây (Quorum

Queues/Streams hiện đại có thể cao hơn đáng kể, nhưng thường vẫn không phải lựa chọn tối ưu cho extreme throughput).

- **Đảm bảo thứ tự (Ordering)**: Kafka đảm bảo thứ tự trên từng partition. RabbitMQ giữ thứ tự FIFO trong một hàng đợi khi chỉ có một consumer hoạt động (*Single Active Consumer*); nếu nhiều competing consumer xử lý song song hoặc đẩy trả message lỗi về hàng đợi (`requeue`), thứ tự hoàn tất thực tế có thể bị đảo (hoặc phải dùng RabbitMQ Streams).
- **Khả năng Replay**: Kafka hỗ trợ chạy lại (replay) từ bất kỳ offset nào. RabbitMQ mặc định không thể replay (trừ khi dùng RabbitMQ Streams bản 3.9+).
- **Cơ chế định tuyến (Routing)**: Kafka định tuyến đơn giản qua topic và partition key. RabbitMQ định tuyến linh hoạt bằng 4 loại exchange và routing key.
- **Hàng đợi ưu tiên (Priority)**: Kafka không hỗ trợ mức ưu tiên. RabbitMQ hỗ trợ tốt hàng đợi ưu tiên (priority queue).
- **Hẹn giờ thông điệp (Delayed message)**: Kafka không có tính năng nguyên bản. RabbitMQ giải quyết qua tổ hợp Dead Letter Exchange + TTL hoặc plugin cộng đồng `rabbitmq_delayed_message_exchange`.
- **Giao thức (Protocol)**: Kafka dùng custom binary protocol tối ưu riêng. RabbitMQ hỗ trợ đa dạng (AMQP, STOMP, MQTT).
- **Bài toán phù hợp**: Kafka dành cho event streaming, stream processing, audit log (ví dụ: lịch sử thao tác của sinh viên). RabbitMQ phù hợp cho task queue, tác vụ hẹn giờ (delayed job), định tuyến phức tạp (ví dụ: phân luồng các lệnh in ấn bảng điểm, gửi email).

Sự khác biệt này quyết định môi trường áp dụng của mỗi broker. Chọn sai công cụ cho bài toán thường dẫn tới việc thiếu đúng tính năng cần dùng, hoặc tắc nghẽn khi tải tăng.

RabbitMQ trong High-throughput

Rocha chia sẻ kinh nghiệm từ một nền tảng thương mại điện tử lớn [5, §3.1.3]: Đội ngũ đã sử dụng RabbitMQ nhiều năm trong môi trường production thông lượng cao. Vấn đề: khi tải đạt đỉnh và thông điệp tích tụ, **toàn bộ cluster bị ảnh hưởng** — các dịch vụ không liên quan cũng bị chậm. Nhìn lại, kịch bản sử dụng thực tế của họ (event streaming, high throughput) phù hợp với durable broker hơn. Bài học: **chọn broker phải dựa trên bài toán thực tế**, không phải theo thói quen.

Lưu ý khi dùng RabbitMQ: Để các worker thực sự chia đều công việc (mô hình Competing Consumers), cần cấu hình `prefetch` nhỏ (ví dụ `prefetch=1`) cho tác vụ kéo dài. `prefetch` giới hạn số message chưa ACK được giao trước cho mỗi consumer: không có giới hạn này, RabbitMQ dồn giao trước (round-robin) hàng loạt message cho worker đang bận trong khi worker khác đã rảnh. Đổi lại, `prefetch` quá nhỏ làm giảm thông lượng với tác vụ ngắn — đây là tham số tuning theo độ dài tác vụ, không phải hằng số bắt buộc.

5.2.4. Khi nào dùng gì?

Bảng 5.1 tóm tắt lựa chọn broker theo từng nhu cầu.

Bảng 5.1: *Khi nào chọn Kafka, khi nào chọn RabbitMQ*

Nhu cầu	Chọn
Event streaming, audit trail, replay	**Kafka**
High throughput (100K+ msg/sec)	**Kafka**
Stream processing (KStreams, ksqlDB)	**Kafka**

Nhu cầu	Chọn
Task/job queue, background worker	**RabbitMQ**
Complex routing (header, topic pattern)	**RabbitMQ**
Priority queue, delayed message	**RabbitMQ**
Cả hai	Kafka cho event backbone + RabbitMQ cho task queue

KBM chọn Kafka cho pipeline xử lý bài nộp (submission) vì cần khả năng phát lại (**replay**) — cho phép chấm lại bài khi logic hệ thống thay đổi, đảm bảo thứ tự (ordering) theo người dùng, và hỗ trợ khả năng mở rộng khi một kỳ thi (contest) có nhiều lượt nộp bài cùng lúc [5, Ch.3]. Phát lại message cũ vẫn phải giữ idempotency; khi chủ động chấm lại bằng logic mới, cần tạo lượt chấm mới với `judgeRunId` mới và chống trùng theo lượt chấm đó. Tuy nhiên, Kafka không phải câu trả lời mặc định cho mọi queue: DevOps Lab dùng **DB queue** — bảng database với status field (`PENDING`, `PROCESSING`, `DONE`) và worker poll định kỳ — cho job vận hành vì cần transactional enqueue đơn giản, dễ quan sát cùng dữ liệu domain và không cần replay event log toàn hệ thống.

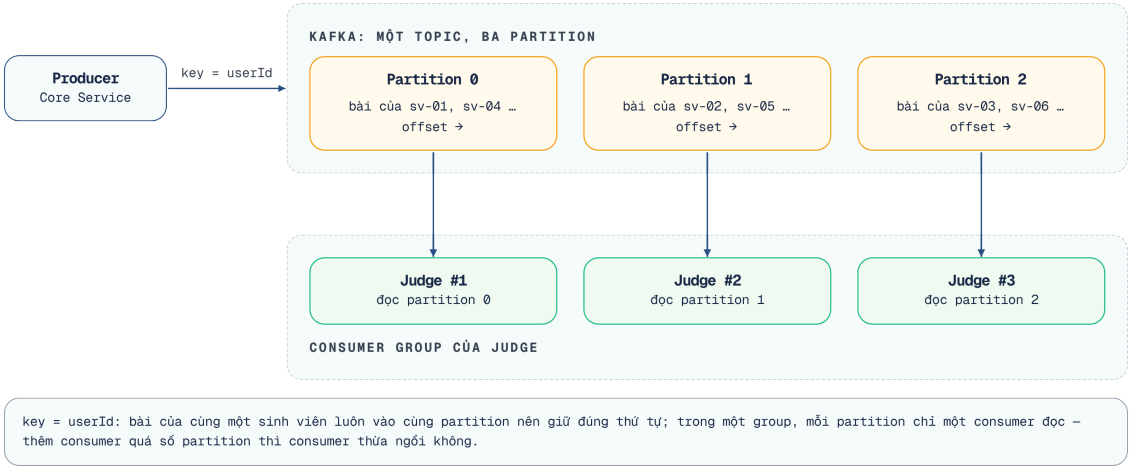
KBM: hai công nghệ, hai bài toán khác nhau

KBM sử dụng Kafka cho luồng xử lý bài nộp (submission pipeline) và hàng đợi cơ sở dữ liệu cho các tác vụ của DevOps Lab — đây là lựa chọn hợp lý xét về khả năng phát lại (replayability) và thông lượng (throughput). Tuy nhiên, việc áp dụng Kafka cho submission (một long-running task) cũng mang rủi ro Head-of-Line blocking; cách giảm rủi ro (thread pool sau consumer hoặc Confluent Parallel Consumer) được bàn ở §5.4 và nằm trong Giai đoạn 3 của lộ trình. Job vận hành cần transactional enqueue, dễ giám sát bằng SQL, và không cần mở rộng lớn → hàng đợi cơ sở dữ liệu là đủ và đơn giản hơn nhiều. **Việc chuẩn hóa cực đoan (over-standardizing) trên Kafka** cho mọi hàng đợi sẽ tạo ra gánh nặng vận hành (operational overhead) không cần thiết.

5.3. Apache Kafka — kiến trúc chi tiết

5.3.1. Các khái niệm cốt lõi

Hình 5.5 cho thấy quan hệ giữa topic, partition và consumer group (minh họa với 3 partition; hiện trạng KBM ở §5.9); từng khái niệm được giải thích ngay sau hình.



Hình 5.5: *Kiến trúc Kafka — Topic, Partition và Consumer Group*

Hệ sinh thái Kafka được cấu thành từ các khái niệm nền tảng sau:

Bảng 5.2: *Các khái niệm cốt lõi của Apache Kafka*

Khái niệm	Mô tả	Ví dụ KBM
Topic	Luồng message theo category	`submissions`, `judge-results`
Partition	Các luồng nhật ký chỉ ghi nối tiếp (append-only log) độc lập trong một topic, cho phép xử lý song song và bảo toàn thứ tự trong từng partition	1 partition cho `submissions` (hiện tại); đề xuất tăng lên 3 khi cần mở rộng
Consumer Group	Nhóm các consumer chia nhau các partition	`judge-group` — hiện chạy 1 instance trên 1 partition (ví dụ mở rộng: nếu chạy 3 instance mà topic vẫn chỉ có 1 partition thì 2 instance sẽ đứng chờ)
Offset	Vị trí của một record trong partition. Consumer lưu riêng vị trí đã commit, giống **dấu kẹp sách** đặt *ngay sau* message cuối cùng đã xử lý thành công	Consumer 1 xử lý xong record có offset 42 → commit position 43; sau restart đọc tiếp từ offset 43
Retention	Thời gian giữ message	7 ngày (default) hoặc vĩnh viễn

Partition key quyết định message vào partition nào. Các thông điệp có cùng khóa (key) sẽ được phân bổ vào cùng một partition, qua đó đảm bảo được thứ tự xử lý. Trong KBM, partition key = `userId` hoặc key theo bài/kỳ thi giúp đảm bảo ordering ở đúng phạm vi nghiệp vụ.

Kleppmann trong [7, Ch.11] giải thích: Kafka kết hợp ưu điểm của database (durable storage, replay) với ưu điểm của messaging (real-time, decoupling) — đây là mô hình **log-based messaging** khác biệt cơ bản với queue-based messaging.

5.4. Producer/Consumer Pattern trong KBM

Trong hệ thống LMS, Producer hoạt động như cổng tiếp nhận bài thi của sinh viên để đẩy vào đường ống xử lý. Ngược lại, Consumer giống như bộ phận chấm thi, lần lượt lấy bài từ hàng đợi ra để chấm.

5.4.1. Kafka Producer

Trong KBM, các bài nộp (submission) được gửi qua Kafka khi sinh viên nộp bài:

Listing 5.1: Kafka Producer — Core Service gửi bài nộp

```
// Producer: Core Service gửi submission vào Kafka
@Service
public class SubmitProducer extends BaseProducerService<SubmitMessage> {

    private static final String TOPIC = "submissions";

    public void sendSubmission(Submission submission) {
        SubmitMessage message = SubmitMessage.builder()
            .submissionId(submission.getId())
            .questionId(submission.getQuestionId())
            .userId(submission.getUserId())
            .sqlContent(submission.getSqlContent())
            .databaseType(submission.getDatabaseType())
            .build();

        // key = userId → đảm bảo ordering per user
        // Rút gọn: mã thật cần kiểm kết quả send;
        // ghi DB và gửi Kafka nguyên tử thì dùng Outbox (Chương 10)
        kafkaTemplate.send(TOPIC, submission.getUserId().toString(), message);
    }
}
```

5.4.2. Kafka Consumer

Phần nâng cao (có thể lướt qua)

Đến hết §5.4, chương phân tích sâu các "bẫy" vận hành Kafka trong production: Head-of-Line Blocking và rebalance. Từ §5.5 trở lại là nội dung cốt lõi. Người đọc mới chỉ cần nắm nguyên lý cốt lõi: consumer đọc message từ topic theo group và xử lý. Các nội dung chuyên sâu về rủi ro vận hành bên dưới có thể quay lại tra cứu khi trực tiếp triển khai hệ thống thực tế.

Judge Service tiêu thụ các bài nộp (submission) và trả kết quả:

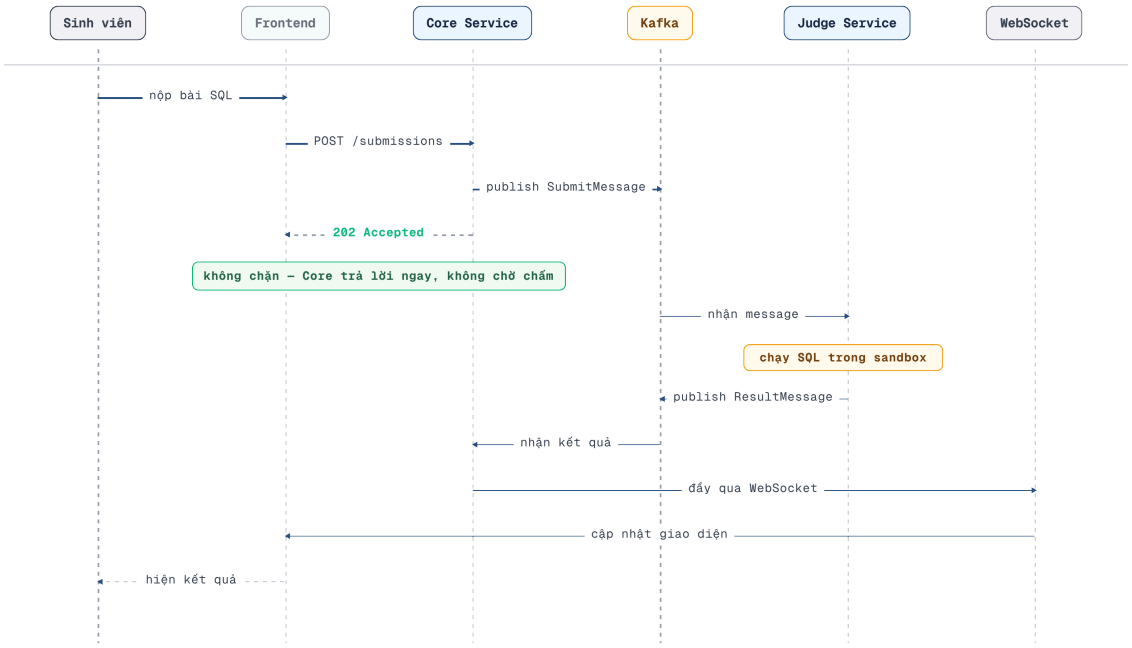
Listing 5.2: Kafka Consumer — cách cài đặt đồng bộ đơn giản (naive) gây Head-of-Line Blocking

```
// Consumer: Judge Service nhận và xử lý (Naive, Blocking)
@KafkaListener(
    topics = "submissions",
    groupId = "judge-group",
    containerFactory = "kafkaListenerContainerFactory"
)
public void processSubmission(SubmitMessage message) {
    // CẢNH BÁO THIẾT KẾ (ANTI-PATTERN): Việc gọi execute() đồng bộ ở đây sẽ chặn
    // thread của Kafka consumer. Nếu task SQL Judge chờ quá lâu và vi phạm cấu hình
    // max.poll.interval.ms, consumer bị coi là đã hỏng và tự rời nhóm, gây ra
    // rebalance loop (Rebalance Storm), làm gián đoạn toàn hệ thống.
    JudgeResult result = sqlExecutorService.execute(message);

    // Gửi kết quả ngược lại qua topic khác
    resultProducer.send("judge-results", message.getSubmissionId(), result);
}
```

5.4.3. Luồng hoàn chỉnh

Sau khi đã xem xét từng thành phần (Producer, Kafka, Consumer) riêng lẻ, sơ đồ dưới đây ghép nối chúng thành trọn luồng: từ lúc sinh viên bấm "Nộp bài" đến khi nhận kết quả chấm trên giao diện.



Hình 5.6: Luồng hoàn chỉnh — từ lúc nộp bài qua Kafka đến kết quả qua WebSocket

Lưu ý: response là **202 Accepted** (không phải 200 OK) — nghĩa là "request đã nhận, đang xử lý". Người dùng nhận kết quả qua thông báo WebSocket, không phải HTTP response.

Thiếu xử lý lỗi trong Kafka pipeline

Một số pipeline của KBM vẫn cần được gia cố (hardening) để đối phó với tình trạng lỗi xử lý thông điệp (message failure): nếu dịch vụ chấm điểm (Judge Service) ngừng hoạt động đột ngột (crash), thông điệp có thể bị mất hoặc bị xử lý lặp tùy thuộc vào cơ chế commit offset. **Giải pháp khuyến nghị** theo [2a, Ch.3]: dùng manual offset commit kết hợp với dead letter topic cho các giao dịch thất bại. **Lộ trình chuyển đổi (Migration path)**: (1) chuyển sang manual commit, (2) áp dụng `@RetryableTopic` cùng dead letter queue, (3) triển khai consumer lag monitoring (giám sát độ trễ tiêu thụ). Lưu ý: retry topic không giữ thứ tự xử lý ban đầu; nếu luồng chấm cần thứ tự theo user thì phải dùng cơ chế retry giữ thứ tự hoặc kiểm soát sequence tại nơi xử lý.

Ngoài lỗi xử lý thông điệp, cách tiêu thụ submission trên Kafka còn một rủi ro về hiệu năng:

Head-of-Line Blocking trong Kafka

Cần phân biệt rõ: dùng Kafka để *nhận* và *đệm* submission (enqueue nhanh, throughput cao, replay) là hợp lý — đó chính là lý do KBM chọn Kafka. Vấn đề chỉ phát sinh khi *xử lý đồng bộ* tác vụ chấm dài (hàng chục giây) *ngay trên consumer thread*: lúc đó nó trở thành anti-pattern nghiêm trọng. Kafka hoạt động theo cơ chế partition, một thông điệp (message) xử lý chậm sẽ

chặn toàn bộ các thông điệp phía sau trong cùng partition (hiện tượng **Head-of-Line Blocking**). Với các tác vụ chạy dài (long-running task) không yêu cầu thứ tự nghiêm ngặt (strict ordering), mô hình Competing Consumers của **RabbitMQ** tỏ ra hiệu quả hơn. Nếu bắt buộc phải dùng Kafka, hệ thống cần triển khai Thread Pool phía sau consumer để tránh nghẽn. (Lưu ý: Nếu tự xây dựng Thread Pool, việc quản lý manual offset commit sẽ rất phức tạp vì các task hoàn thành không theo thứ tự, dễ gây mất dữ liệu khi service crash. Do đó, nên ưu tiên dùng các thư viện chuyên dụng đã xử lý sẵn bài toán này như **Confluent Parallel Consumer**. Từ Kafka 4.2, share group (KIP-932) cũng là một lựa chọn: nhiều consumer xử lý đồng thời trên cùng partition và xác nhận từng record, đổi lại không còn thứ tự theo partition).

Lưu ý (Rebalance Storm): Khi consumer xử lý một batch các long-running task, tổng thời gian rất dễ vượt quá giới hạn `max.poll.interval.ms` (mặc định là 5 phút). Khi đó, consumer bị coi là đã hỏng và tự rời nhóm, nhóm phải rebalance, trong khi tác vụ thực tế vẫn đang chạy. Điều này dẫn đến vòng lặp rebalance vô tận (Endless Rebalance). Để khắc phục, cần tuning cấu hình: tăng `max.poll.interval.ms` hoặc giảm `max.poll.records` (số lượng message mỗi lần poll) để đảm bảo thời gian xử lý batch luôn nằm trong ngưỡng an toàn.

5.5. Delivery Guarantees & Idempotency

Quản lý thông điệp giống việc thu bài thi: mỗi bài phải được ghi nhận đúng một lần, không sót, không chấm hai lần.

5.5.1. Ba cấp độ đảm bảo

Đây là một trong những khái niệm quan trọng nhất trong messaging — và cũng dễ bị hiểu sai nhất [7, Ch.11]. Trong hệ thống phân tán, cơ chế giao nhận thông điệp có ba cấp độ đảm bảo:

- **At-most-once (gửi tối đa một lần)**: Ưu tiên tốc độ, đổi lại là rủi ro mất thông điệp.
- **At-least-once (gửi ít nhất một lần)**: Đảm bảo độ tin cậy, nhưng có thể sinh ra thông điệp trùng lặp — consumer buộc phải xử lý idempotent.
- **Exactly-once (xử lý đúng một lần)**: Cấp độ cao nhất, đổi lại là chi phí hiệu năng đắt đỏ.

Kleppmann trong [7, Ch.11] phân tích: **exactly-once semantics** trong thực tế là *effectively-once* — đạt được bằng cách kết hợp at-least-once delivery + idempotent processing. Kafka transactions (từ bản 0.11) hỗ trợ exactly-once **trong Kafka**, nhưng không xuyên hệ thống (Kafka → DB).

5.5.2. Idempotency — thiết kế để chịu được duplicate

Khi sử dụng chiến lược at-least-once (phổ biến nhất), phía tiêu thụ (consumer) *có thể* nhận cùng một thông điệp (message) nhiều lần. Mã nguồn phải đảm bảo idempotency — xử lý nhiều lần cho cùng kết quả [5, Ch.8]:

Listing 5.3: *Idempotent consumer — kiểm tra trước khi xử lý*

```
// SAI: Không idempotent — chấm trùng = điểm sai
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
    // Nếu message trùng → score bị cộng dồn hoặc ghi đè không đúng state
}

// ĐÚNG: Idempotent — check trước khi xử lý
```

```
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    if (submissionRepository.isAlreadyJudged(msg.getSubmissionId())) {
        log.info("Submission {} already judged, skipping", msg.getSubmissionId());
        return; // Skip duplicate
    }
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
}
```

Rocha trong [5, Ch.8] nhấn mạnh: event consumption phải **idempotent** — xử lý nhiều lần vẫn cho cùng kết quả — và, khi domain cho phép, nên thiết kế thêm tính commutative/associative để chịu được thứ tự xáo trộn. Với các luồng bắt buộc đúng thứ tự, hệ thống nên dựa vào ordering theo partition key (mục trên) và version/sequence của aggregate để từ chối hoặc trì hoãn event sai thứ tự, thay vì cố ép mọi phép xử lý phải có tính giao hoán.

Race Window trong Check-then-write

Mẫu `isAlreadyJudged()` rồi `updateScore()` ở trên đủ để minh họa ý tưởng, nhưng *chưa* an toàn tuyệt đối khi hai bản sao consumer xử lý cùng một message song song: cả hai có thể cùng vượt qua bước kiểm tra trước khi bất kỳ bên nào kịp ghi. Để idempotency chắc chắn, cần dựa vào **ràng buộc ở tầng database** — ví dụ `UNIQUE constraint` trên `submissionId`, thao tác `UPSERT`/`INSERT ... ON CONFLICT DO NOTHING`, hoặc một bảng "đã xử lý" (processed-message table) trong cùng transaction với việc ghi kết quả.

Trong KBM, idempotency đặc biệt quan trọng ở SQL Judge: submission có thể được retry qua Kafka/queue, và bộ điều phối `kbm-judge-sql` có thể gọi lại cùng một worker `kbm-sandbox-*` sau timeout. Mỗi command nên có `submissionId` hoặc `judgeRunId` ổn định; consumer/worker kiểm tra trạng thái trước khi ghi kết quả. Nếu đã chấm xong, hệ thống trả về kết quả hiện có hoặc bỏ qua message, thay vì tạo thêm một lần chấm mới.

5.6. Event Schema Design

Mọi thông báo gửi đi trong trường học đều cần tuân thủ một biểu mẫu chuẩn. Trong kiến trúc sự kiện, việc thống nhất cấu trúc dữ liệu chung này là yêu cầu bắt buộc, giúp các dịch vụ đọc hiểu nhau dễ dàng mà không vấp phải lỗi tương thích.

5.6.1. Cấu trúc một event

Theo tinh thần thiết kế của Rocha [5, Ch.8], mỗi event nên tách thành hai phần rõ rệt:

Listing 5.4: *Cấu trúc event với metadata và payload*

```
{
  "metadata": {
    "eventId": "e7d3f2a1-...",
    "eventType": "SubmissionJudged",
    "schemaVersion": "1.0",
    "occurredAt": "2026-03-10T14:30:00Z",
    "source": "kbm-judge-sql",
    "correlationId": "sub-456",
    "causationId": "cmd-sub-456"
  },
```

```
  "payload": {
    "submissionId": "sub-456",
    "questionId": "q-789",
    "userId": "user-001",
    "result": "CORRECT",
    "executionTimeMs": 245
  }
}
```

Một event tiêu chuẩn cần tách biệt rõ ràng giữa hai phần: **metadata** cung cấp siêu dữ liệu phục vụ chống trùng lặp (deduplication), quản lý phiên bản (versioning) và đối chiếu ngữ cảnh qua bộ định danh chuẩn — `eventId` (định danh bất biến của thông điệp), `correlationId` (gom nhóm toàn bộ tiến trình nghiệp vụ, ví dụ một lượt nộp bài), và `causationId` (ID của lệnh hoặc sự kiện trực tiếp sinh ra thông điệp này); ngữ cảnh truy vết (`traceparent`/`tracestate` chuẩn W3C) không nằm trong envelope mà đi trong Kafka header, do instrumentation tự truyền ở tầng transport (xem Ch.11); trong khi đó, **payload** chứa dữ liệu nghiệp vụ (business data) cụ thể của domain. Trong hệ sinh thái Cloud Native, cấu trúc tách biệt giữa metadata và payload này đã được chuẩn hóa thành tiêu chuẩn công nghiệp **CloudEvents** (do CNCF quản lý), giúp dễ tích hợp với các công cụ trung gian.

5.6.2. Bốn loại message

Rocha phân loại message thành 4 loại, mỗi loại có mục đích và naming convention riêng [5, §3.1.4]:

__Bảng 5.3:__ Bốn loại message trong kiến trúc hướng sự kiện

Loại	Mô tả	Cách đặt tên	Ví dụ KBM	Phân phối
Command	Yêu cầu thực hiện hành động — *có thể bị từ chối*	Verb imperative	`JudgeSubmission`, `SendNotification`	Point-to-point
Event	Thông báo sự kiện đã xảy ra — *sự thật, không thể reject*	Past participle	`SubmissionJudged`, `ContestStarted`	Pub/sub
Document	Snapshot toàn bộ entity khi thay đổi — *full state, hiện thân của Event-Carried State Transfer Pattern*	Noun	`SubmissionDocument`, `UserDocument`	Pub/sub
Query	Yêu cầu thông tin, không thay đổi state	Noun + request	Thường qua HTTP (broker có thể dùng Request-Reply Pattern nhưng ít phổ biến)	Sync (hoặc RPC)

5.6.3. Schema evolution

Quá trình tiến hóa cấu trúc dữ liệu (Schema evolution) diễn ra tương tự như việc nâng cấp API versioning đã đề cập ở Chương 3. Việc thay đổi event schema cũng đòi hỏi một chiến lược tiến hóa nhất quán [7, Ch.4]. Hãy hình dung biểu mẫu đăng ký học phần của sinh viên qua các năm:

- **Thêm field optional (Tương thích ngược)**: Bổ sung một cột "Nguyện vọng 2" không bắt buộc. Biểu mẫu cũ vẫn dùng được, biểu mẫu mới mang thêm thông tin — một thay đổi an toàn (✓).
- **Bỏ field mà consumer còn dùng (Phá vỡ cấu trúc)**: Xóa bỏ cột "Mã số sinh viên". Các hệ thống (consumer) cũ vẫn phụ thuộc vào thông tin này để xử lý hồ sơ, dẫn đến lỗi toàn hệ thống (×).
- **Đổi field type (Phá vỡ cấu trúc)**: Thay đổi mã môn học từ dạng số sang chuỗi ký tự dài. Các hệ thống cũ vốn lưu mã môn là số nguyên sẽ lập tức bị gián đoạn (×).

Giải pháp được khuyến nghị là triển khai một **Schema Registry** (như Confluent Schema Registry hoặc AWS Glue) hoạt động như một kho schema trung tâm. Giống như việc Phòng Đào tạo phải

kiểm duyệt và đóng dấu mọi thay đổi trên mẫu đơn trước khi phát hành, mỗi sự thay đổi về schema đều được hệ thống tự động kiểm chứng mức độ tương thích trước khi cho phép ban hành (publish).

KBM thiếu event schema chuẩn

Các thông điệp trong KBM là các đối tượng Java thuần (POJO) được tuần tự hóa (serialize) dưới định dạng JSON — không có metadata chuẩn (eventId, occurredAt, correlationId), không có schema registry, không có version. Khi Judge Service thay đổi định dạng thông điệp, Core Service có thể gặp sự cố (crash) nếu chưa được cập nhật. **Lộ trình chuyển đổi**: (1) thêm metadata wrapper cho tất cả thông điệp Kafka, (2) sử dụng `eventId` để kiểm tra idempotency, (3) cân nhắc Avro + Schema Registry khi quy mô đội ngũ phát triển mở rộng.

5.7. Kafka vs DB Queue — chọn broker theo bài toán

Không phải mọi luồng xử lý bất đồng bộ đều cần Kafka. Việc chọn đúng cơ chế hàng đợi phụ thuộc vào yêu cầu cụ thể của từng bài toán trong hệ thống.

__Bảng 5.4:__ Chọn Kafka hay DB queue theo bài toán

Bài toán	Lựa chọn	Lý do
SQL submission / contest	Kafka	Cần throughput, ordering theo key, replay và consumer group
SQL Judge worker dispatch (`kbm-judge-sql` → `kbm-sandbox-*`)	Kafka hoặc DB queue + worker	Mỗi tác vụ (job) mang một định danh (`submissionId`/`judgeRunId`) ổn định, thuận tiện cho việc thử lại (retry) và đảm bảo idempotency; các worker theo từng DBMS (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`…) xử lý độc lập
DevOps Lab operations	DB queue	Job gắn chặt với state trong database, cần transaction enqueue cùng thay đổi domain
Audit/learning analytics	Kafka	Nhiều consumer có thể đọc cùng event stream

Việc sử dụng DB queue không làm giảm đi tính chất microservices của hệ thống so với Kafka. Nó là lựa chọn hợp lý khi lưu lượng (volume) vừa phải, số lượng phía tiêu thụ (consumer) ít, và yêu cầu quan trọng nhất là tính nguyên tử (atomicity) giữa trạng thái nghiệp vụ và bản ghi tác vụ. Kafka tỏ ra vượt trội hơn khi hệ thống cần phân tán thông điệp (fan-out), phát lại dữ liệu (replay), phân vùng (partitioning) và xử lý luồng (stream processing).

5.8. WebSocket — Real-time Notifications

Thay vì để sinh viên phải liên tục làm mới trang web để xem điểm thi, hệ thống cần một kênh liên lạc chủ động. WebSocket mở một kết nối hai chiều, cho phép kết quả hiện lên ngay khi Judge Service chấm xong bài.

5.8.1. Bài toán

Sau khi Judge Service chấm xong, kết quả cần đến tay sinh viên. Có ba phương án:

- **Polling**: Client hỏi lại định kỳ. Dễ cài đặt, nhưng lãng phí băng thông và độ trễ cao.
- **Long polling**: Client giữ kết nối chờ đến khi server có dữ liệu mới. Khắc phục được độ trễ, nhưng khó mở rộng.
- **Server-Sent Events (SSE)**: Kết nối một chiều từ server tới client trên HTTP. Đơn giản hơn WebSocket khi không cần chiều ngược lại.

- **WebSocket**: Kết nối hai chiều, server chủ động đẩy dữ liệu thời gian thực. Đổi lại là chi phí quản lý trạng thái kết nối.

KBM sử dụng **STOMP over WebSocket** — sub-protocol messaging chạy trên WebSocket, triển khai bằng Spring WebSocket (có thể bật thêm SockJS fallback qua `.withSockJS()` khi cần hỗ trợ trình duyệt cũ; khi đó client phải dùng `webSocketFactory` thay cho `brokerURL`):

__Listing 5.5:__ WebSocket server — đẩy kết quả chấm bài qua STOMP

```
// Server-side: push kết quả chấm bài
@Service
public class NotificationService {
    private final SimpMessagingTemplate messagingTemplate;

    public NotificationService(SimpMessagingTemplate messagingTemplate) {
        this.messagingTemplate = messagingTemplate;
    }

    public void notifyJudgeResult(UUID userId, JudgeResult result) {
        messagingTemplate.convertAndSendToUser(
            userId.toString(),
            "/queue/notifications",
            new JudgeNotification(result)
        );
    }
}
```

Phía trình duyệt, client đăng ký (subscribe) vào hàng đợi riêng của người dùng để nhận thông báo (Listing 5.6):

__Listing 5.6:__ WebSocket client — đăng ký (subscribe) nhận kết quả theo thời gian thực

```
// Client-side: subscribe nhận kết quả
const stompClient = new StompJs.Client({
    brokerURL: 'wss://<gateway-host>/ws'
});

stompClient.onConnect = () => {
    stompClient.subscribe('/user/queue/notifications', (message) => {
        const result = JSON.parse(message.body);
        showNotification(`Kết quả: ${result.status}`);
    });
};
stompClient.activate();
```

WebSocket phù hợp cho **thông báo ở cuối pipeline** (kết quả chấm bài, cập nhật bảng xếp hạng trong kỳ thi) nhưng không thay thế Kafka cho **messaging giữa các service** — hai vai trò khác nhau. Khi service chạy nhiều instance, kết nối của sinh viên chỉ nằm trên một instance, nên thông báo phải đi qua một broker chung để tới đúng nơi.

5.9. Case study: KBM

Kỳ thi tính giờ trên KBM là phép thử rõ nhất cho đường ống bất đồng bộ. Tải không lớn nếu tính trung bình, nhưng dồn cục vào vài phút cuối giờ làm bài.

5.9.1. Bài toán kỳ thi

Trong chế độ thi, khoảng 200 sinh viên đồng thời nộp bài trong thời gian giới hạn (60–120 phút). Yêu cầu:

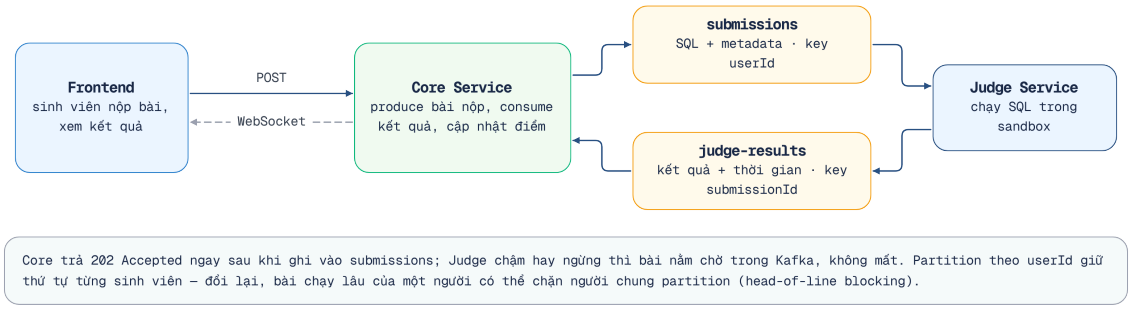
- Thông lượng cao: Xử lý hàng trăm lượt nộp bài mỗi phút vào cao điểm cuối giờ (kịch bản chất lượng §1.5)
- Thứ tự công bằng: Đảm bảo thứ tự xử lý cho từng user
- Phản hồi thời gian thực: Sinh viên thấy kết quả ngay
- Cập nhật bảng xếp hạng: Bảng xếp hạng cập nhật liên tục

⚠ Cạm bẫy: key co-location

Yêu cầu "thứ tự công bằng" (partition theo `userId`) lại có mặt trái: khi nhiều user rơi vào cùng một partition (key co-location — hệ quả bình thường của partition theo key, không phải hash collision), bài nộp chậm của người này chặn người kia. Sinh viên A nộp một truy vấn chạy chậm (ví dụ JOIN không điều kiện trên bảng lớn) mất 30 giây để xử lý sẽ chặn đứng bài nộp chỉ cần 1 giây của sinh viên B nếu cả hai thông điệp rơi vào cùng một phân vùng (partition). Đây chính là Head-of-Line Blocking đã phân tích ở trên.

5.9.2. Kiến trúc pipeline 2 Kafka topic

Hình 5.7 mô tả pipeline chấm bài trong chế độ thi.



Hình 5.7: *Kiến trúc pipeline 2 Kafka topic chấm bài trong chế độ thi*

Pipeline chấm bài dùng hai Kafka topic: `submissions` đưa bài nộp tới Judge, `judge-results` đưa kết quả về Core; chặng cuối đẩy thông báo theo `userId` tới trình duyệt đi qua WebSocket (§5.8), không tạo thêm Kafka topic thứ ba.

Bảng 5.5: *Chi tiết các Kafka topic trong pipeline*

Topic	Producer	Consumer	Message	Key
`submissions`	Core Service	Judge Service	SQL + metadata	`userId`
`judge-results`	Judge Service	Core Service	Result + timing	`submissionId`

5.9.3. Phân tích các vấn đề

Đối chiếu với các giải pháp khuyến nghị, cấu hình hiện tại còn năm điểm yếu:

Bảng 5.6: Phân tích vấn đề Kafka pipeline trong KBM

#	Vấn đề	Hiện trạng	Giải pháp khuyến nghị [2a]
1	**Commit offset chưa gắn với xử lý**	Offset có thể được commit trước khi xử lý xong, nhất là khi xử lý ở luồng khác → message loss	Manual commit sau khi xử lý thành công
2	**Không có DLT**	Thông điệp lỗi bị thử lại (retry) vô hạn hoặc bị mất	Cấu hình Dead Letter Topic cho các thông điệp lỗi
3	**Không idempotent**	Thông điệp trùng lặp (duplicate) có thể tạo ra kết quả sai	Kiểm tra `submissionId` trước khi chấm điểm (judge)
4	**Thiếu monitoring**	Không nắm được độ trễ của phía tiêu thụ (consumer lag)	Thiết lập Kafka consumer lag metric và cảnh báo
5	**Single partition**	Tất cả bài nộp (submission) bị đẩy vào 1 partition dẫn đến tắc nghẽn (bottleneck)	Tăng số partition và giữ nguyên `key = userId` đang dùng để phân tán tải theo sinh viên

5.9.4. Lộ trình củng cố pipeline

Quá trình chuyển đổi và củng cố pipeline cần được chia thành các giai đoạn rõ ràng. Trong **Giai đoạn 1 — Độ tin cậy** (ưu tiên cao), hệ thống cần đảm bảo không mất mát dữ liệu bằng cách sử dụng manual offset commit kết hợp `@RetryableTopic` (với 3 lần thử lại) và cấu hình Dead Letter Topic. Đồng thời, phải bổ sung bước kiểm tra idempotency thông qua `submissionId`.

Chuyển sang **Giai đoạn 2 — Observability** (cũng ưu tiên cao), trọng tâm là khả năng giám sát. Hệ thống cần triển khai Kafka consumer lag monitoring sử dụng Kafka Exporter và Prometheus để theo dõi độ trễ. Việc truyền `traceId` và `correlationId` xuyên suốt pipeline (từ submission đến judge, result, và notification) cũng bắt buộc để dễ dàng truy vết (tracing).

Cuối cùng, ở **Giai đoạn 3 — Hiệu năng** (khi cần mở rộng quy mô), hệ thống bắt buộc phải tách biệt (decouple) Consumer Thread và Execution Thread. Consumer chỉ nên chịu trách nhiệm lấy dữ liệu rồi đẩy vào Local Thread Pool để xử lý thực tế (kèm cơ chế quản lý offset cẩn trọng để tránh mất dữ liệu do out-of-order execution), hoặc ưu tiên sử dụng thư viện chuyên dụng như Confluent Parallel Consumer. Chỉ tăng số lượng partition (ví dụ lên 3) sẽ không giải quyết được tắc nghẽn nếu nhiều bài nộp chạy chậm cùng lúc. Muốn tăng thông lượng, còn phải thêm các instance Judge vào consumer group — với điều kiện số partition được tăng tương ứng: số consumer hoạt động đồng thời trong một group không thể vượt quá số partition của topic, instance dư ra sẽ chỉ đứng chờ (idle). Tăng partition cũng đổi ánh xạ key → partition cho message ghi sau đó; nếu cần giữ thứ tự theo user, phải kiểm soát thời điểm chuyển đổi, chẳng hạn tạm dừng producer và xử lý hết backlog trước. Số partition lớn hơn còn làm rebalance chậm hơn và tốn tài nguyên broker hơn.

⚠ Sai lầm thường gặp khi làm việc với Message Broker

- **Auto-commit offset khi xử lý chạy ở luồng khác**: Consumer xác nhận (acknowledge) thông điệp trước khi xử lý xong. Giống như việc chuyên viên đào tạo đóng dấu "đã xử lý" lên đơn đăng ký tín chỉ trước khi thực sự cập nhật vào hệ thống; nếu hệ thống tắt nguồn đột ngột lúc đó, đơn sẽ bị quên lãng vĩnh viễn. *Phòng tránh*: Dùng manual offset commit — chỉ commit *sau khi* processing thành công.
- **Không có Dead Letter Topic**: Thông điệp lỗi bị thử lại (retry) vô hạn hoặc làm tắc nghẽn (block) toàn bộ pipeline. Một đơn xin bảo lưu bị lỗi định dạng có thể chặn đứng mọi đơn từ khác xếp sau nó (hiện tượng poison pill). *Phòng tránh*: Cấu hình DLT (`@RetryableTopic` trong

Spring Kafka) — chuyển các message lỗi sau N lần thử sang một hàng đợi riêng để xử lý thủ công.

- **Không thiết kế idempotent consumer**: Giả định mỗi message chỉ đến đúng một lần. Khi Kafka rebalance, một bài tập có thể bị chấm hai lần, khiến sinh viên nhận được hai kết quả khác nhau. *Phòng tránh*: Kiểm tra trạng thái trước khi xử lý, nếu đã chấm rồi thì bỏ qua (§5.5).
- **Chọn broker vì quen thuộc, không vì bài toán**: Sử dụng RabbitMQ cho luồng sự kiện (event streaming) chỉ vì đội ngũ phát triển đã quen thuộc với công nghệ này. Hậu quả thường không phải quá tải mà là thiếu thứ cần dùng: RabbitMQ xóa thông điệp sau khi consumer xác nhận nên không phát lại được luồng sự kiện để chấm lại hay cho consumer mới. *Phòng tránh*: Đánh giá bài toán trước (§5.2) — chọn Kafka cho event streaming, RabbitMQ cho task queue.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Cơ chế hoạt động của Message Broker**: message-broker.html
- **Idempotency và retry**: idempotency-retry.html

5.10. Tổng kết

Giao tiếp bất đồng bộ làm nhẹ ba giới hạn cốt lõi của sync: temporal coupling, cascading failure, và độ trễ. Hai trường phái message broker — durable (Kafka) và ephemeral (RabbitMQ) — phục vụ các bài toán khác nhau, và nhiều hệ thống dùng cả hai.

Kafka với mô hình event log (durable, replayable, high-throughput) trở thành nền tảng phổ biến nhất cho event-driven architecture. RabbitMQ với smart routing và flexible messaging phù hợp cho task queue, delayed job, và các mẫu định tuyến phức tạp. Partition, consumer group, và exchange type cho phép mỗi broker mở rộng theo cách riêng.

Delivery guarantees (at-most/at-least/exactly-once) và idempotency là hai khái niệm không thể tách rời. Thiết kế event schema chuẩn — với metadata, versioning, và correlation — là đầu tư cần thiết giúp hệ thống dễ dàng gỡ lỗi (debug) và tiến hóa (evolve).

Phân tích KBM cho thấy pipeline Kafka hoạt động nhưng thiếu nhiều giải pháp khuyến nghị quan trọng: xử lý lỗi, idempotency, giám sát. Đồng thời, DevOps Lab nhắc lại một bài học thực tế: có những job queue nên nằm gần database thay vì ép qua Kafka. Đây là nợ kỹ thuật (technical debt) và sự đánh đổi (trade-off) phổ biến khi các đội ngũ nhỏ ưu tiên tính năng trước độ tin cậy (reliability).

Ở Chương 6, chúng ta sẽ đối mặt với một bài toán khó của hệ phân tán: **giao dịch phân tán (distributed transaction)**. Khi dữ liệu nằm ở nhiều service khác nhau, làm thế nào để bảo đảm tính nhất quán? Saga pattern sẽ là câu trả lời.

5.11. Đọc thêm

Sách tham khảo chính:

1. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.1: Why EDA; Ch.3: Kafka; Ch.8: Event Schema Design

2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Async Messaging, Transactional Outbox
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.11: Stream Processing, Event Logs

Sách bổ trợ:

4. [3] Mitra & Nadareishvili, *Microservices: Up and Running* — Ch.4: Event-Driven Communication

Nguồn trực tuyến:

- Apache Kafka documentation — kafka.apache.org/documentation
- Confluent Schema Registry — docs.confluent.io/platform/current/schema-registry
- Spring Kafka reference — docs.spring.io/spring-kafka

Bài tập: 5-0, 5-1, 5-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running: A Step-by-Step Guide to Building a Microservices Architecture*. O'Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.